

Phân loại bình luận độc hại

Vũ Công Tuấn Dương-B22DCKH024
Nhóm 20

Khoa CNTT1 – PTIT

Ngày 7 tháng 6 năm 2026

Mục lục

- ① Giới thiệu
- ② Cơ sở lý thuyết
 - LSTM
 - BiLSTM
 - GRU
 - RCNN
 - Transformer
 - Attention
- ③ Phương pháp tối ưu
- ④ Kết quả và nhận xét

- 1 Giới thiệu
- 2 Cơ sở lý thuyết
- 3 Phương pháp tối ưu
- 4 Kết quả và nhận xét

Mục tiêu đề tài

- **Bài toán:** Phân loại đa nhãn (Jigsaw) & đa lớp (HateXplain) bình luận độc hại
 - Jigsaw: 6 nhãn — toxic, severe_toxic, obscene, threat, insult, identity_hate
 - HateXplain: 3 nhãn — normal, offensive, hatespeech
- **Cài đặt từ gốc (from scratch)** — 3 kiến trúc chính:
 - RCNN, Attention BiLSTM, Transformer (Pre-LN, 4 tầng)
 - Được chọn từ 7 mô hình ban đầu (LSTM, BiLSTM, GRU,...)
- **Tối ưu hóa:** AdamW, Gradient Clipping, Warmup Cosine LR, Class Weights, Focal Loss, Masked Pooling, AMP, GloVe 300D
- **Đánh giá chéo:** Huấn luyện độc lập trên Jigsaw và HateXplain

Tập dữ liệu

- **Nguồn:** Jigsaw Toxic Comment Classification Challenge (Kaggle)
- **Tổng số mẫu:** 159,571 bình luận
- **Chia tập:** 80% train / 20% validation (stratified split)
- **Đặc điểm:** Mất cân bằng lớp nghiêm trọng
- **Validation set:** 31,915 mẫu

Nhãn	Tỷ lệ (%)	Class Weight (đã clip)	Số mẫu
toxic	9.4	9.43	15,000
severe_toxic	1.0	50.00 (clip)	1,600
obscene	5.0	17.89	8,000
threat	0.2	50.00 (clip)	320
insult	4.8	19.26	7,600
identity_hate	0.9	50.00 (clip)	1,400

Tập dữ liệu HateXplain

- **Nguồn:** HateXplain (AAAI 2021)
- **Nền tảng:** Twitter, Reddit — bình luận thực tế
- **Tổng số mẫu:** 20,109 bình luận (3 lớp)
- **Chia tập:** 80% train / 10% validation / 10% test
- **3 nhãn:** normal (52%), offensive (36%), hatespeech (12%)
- **Đặc điểm:** Mất cân bằng lớp, ngôn ngữ tự nhiên đa dạng

Nhãn	Số mẫu	Tỷ lệ
normal	10,457	52%
offensive	7,239	36%
hatespeech	2,413	12%

① Giới thiệu

② Cơ sở lý thuyết

- LSTM
- BiLSTM
- GRU
- RCNN
- Transformer
- Attention

③ Phương pháp tối ưu

④ Kết quả và nhận xét

① Giới thiệu

② Cơ sở lý thuyết

- LSTM
- BiLSTM
- GRU
- RCNN
- Transformer
- Attention

③ Phương pháp tối ưu

④ Kết quả và nhận xét

Long Short-Term Memory (LSTM)

Vấn đề của RNN truyền thống: Vanishing/Exploding Gradient khi xử lý chuỗi dài.

Giải pháp: LSTM sử dụng 3 cổng (gates) để điều khiển luồng thông tin:

- **Forget gate (f_t):** Quyết định thông tin nào giữ lại từ c_{t-1}
- **Input gate (i_t):** Quyết định thông tin mới nào lưu vào cell state
- **Output gate (o_t):** Quyết định thông tin nào xuất ra hidden state

$$f_t = \sigma(W_{hf}h_{t-1} + W_{xf}x_t + b_f)$$

$$i_t = \sigma(W_{hi}h_{t-1} + W_{xi}x_t + b_i)$$

$$\tilde{c}_t = \tanh(W_{hc}h_{t-1} + W_{xc}x_t + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$o_t = \sigma(W_{ho}h_{t-1} + W_{xo}x_t + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

① Giới thiệu

② Cơ sở lý thuyết

- LSTM
- **BiLSTM**
- GRU
- RCNN
- Transformer
- Attention

③ Phương pháp tối ưu

④ Kết quả và nhận xét

Bidirectional LSTM (BiLSTM)

Ý tưởng: Xử lý chuỗi theo **cả hai hướng** để nắm bắt ngữ cảnh quá khứ và tương lai.

- **Forward LSTM:** Đọc từ trái $\rightarrow$ phải, nắm bắt ngữ cảnh trước
- **Backward LSTM:** Đọc từ phải $\rightarrow$ trái, nắm bắt ngữ cảnh sau
- **Hidden state cuối:** Nối (concat) hai hướng

$$\begin{aligned}\overrightarrow{h}_t &= \text{LSTM}_{\rightarrow}(x_t, \overrightarrow{h}_{t-1}), & \overleftarrow{h}_t &= \text{LSTM}_{\leftarrow}(x_t, \overleftarrow{h}_{t+1}) \\ h_t &= [\overrightarrow{h}_t; \overleftarrow{h}_t]\end{aligned}$$

Ưu điểm: Hiểu ngữ cảnh toàn cục tốt hơn so với LSTM đơn hướng.

① Giới thiệu

② Cơ sở lý thuyết

- LSTM
- BiLSTM
- **GRU**
- RCNN
- Transformer
- Attention

③ Phương pháp tối ưu

④ Kết quả và nhận xét

Gated Recurrent Unit (GRU)

Đặc điểm: Biến thể đơn giản hóa của LSTM, chỉ có **2 cổng**:

- **Update gate (z_t):** Kết hợp vai trò của forget + input gate
- **Reset gate (r_t):** Kiểm soát thông tin quá khứ cần "quên"

$$z_t = \sigma(W_{hz}h_{t-1} + W_{xz}x_t + b_z)$$

$$r_t = \sigma(W_{hr}h_{t-1} + W_{xr}x_t + b_r)$$

$$\tilde{h}_t = \tanh(W_{hn}(r_t \odot h_{t-1}) + W_{xn}x_t + b_n)$$

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

Ưu điểm: Ít tham số hơn LSTM $\rightarrow$ huấn luyện nhanh hơn, ít nguy cơ overfitting.

① Giới thiệu

② Cơ sở lý thuyết

- LSTM
- BiLSTM
- GRU
- **RCNN**
- Transformer
- Attention

③ Phương pháp tối ưu

④ Kết quả và nhận xét

Recurrent Convolutional Neural Network (RCNN)

Ý tưởng: Kết hợp ưu điểm của RNN (ngữ cảnh tuần tự) và CNN (trích xuất đặc trưng cục bộ).

Kiến trúc:

- ➊ **Forward + Backward RNN:** Thu thập ngữ cảnh trái và phải cho mỗi từ
- ➋ **Concatenate:** Nối [ngữ cảnh trái + embedding từ + ngữ cảnh phải]
- ➌ **Fusion layer:** $\tanh(W \cdot [c_L; x; c_R] + b)$ – tích hợp đặc trưng
- ➍ **Max pooling:** Lấy đặc trưng quan trọng nhất trên toàn bộ chuỗi
- ➎ **Fully connected:** Phân loại đầu ra

Ưu điểm: Max pooling giúp tập trung vào các từ khóa quan trọng nhất, bất kể vị trí.

① Giới thiệu

② Cơ sở lý thuyết

- LSTM
- BiLSTM
- GRU
- RCNN
- **Transformer**
- Attention

③ Phương pháp tối ưu

④ Kết quả và nhận xét

Transformer Encoder (Self-Attention)

Khác biệt cốt lõi: Không dùng RNN → xử lý song song toàn bộ chuỗi.

Self-Attention: Mỗi từ "chú ý" đến tất cả từ khác trong câu:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Multi-Head Attention: Chạy nhiều attention song song với các không gian con khác nhau:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Positional Encoding: Bổ sung thông tin vị trí vì không có thứ tự tự nhiên:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Kiến trúc Transformer Encoder Block

Mỗi block gồm 2 thành phần chính:

① **Multi-Head Self-Attention:**

- Tính attention giữa tất cả cặp từ
- Masking với attention mask (bỏ qua padding tokens)

② **Feed-Forward Network:**

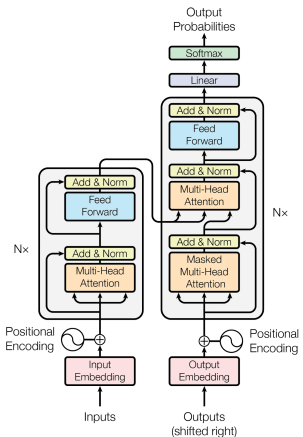
- 2 lớp tuyến tính: Linear $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ Linear
- Áp dụng độc lập cho từng vị trí

Residual connection + Layer Normalization:

$$x' = \text{LayerNorm}(x + \text{Dropout}(\text{Attention}(x)))$$

$$x'' = \text{LayerNorm}(x' + \text{Dropout}(\text{FFN}(x')))$$

Transformer Encoder Block (sơ đồ)



Hình: Sơ đồ kiến trúc Transformer

① Giới thiệu

② Cơ sở lý thuyết

- LSTM
- BiLSTM
- GRU
- RCNN
- Transformer
- **Attention**

③ Phương pháp tối ưu

④ Kết quả và nhận xét

Cơ chế Attention

Ý tưởng: Thay vì dùng hidden state cuối cùng hoặc mean pooling, gán **trọng số khác nhau** cho từng time step.

Self-Attention (Bahdanau-style):

$$e_t = \tanh(W_a h_t + b_a)$$

$$\alpha_t = \text{softmax}(v^T e_t)$$

$$c = \sum_{t=1}^T \alpha_t h_t$$

- h_t : hidden state tại time step t
- α_t : trọng số attention (tổng = 1)
- c : context vector – trọng số của toàn bộ chuỗi

Ưu điểm: Mô hình tự động học được từ/cụm từ nào quan trọng nhất cho việc phân loại.

- 1 Giới thiệu
- 2 Cơ sở lý thuyết
- 3 Phương pháp tối ưu
- 4 Kết quả và nhận xét

Class-Weighted Loss

Vấn đề: Dữ liệu mất cân bằng nghiêm trọng (threat chỉ 0.2%, toxic 9.4%).

Giải pháp: BCEWithLogitsLoss với trọng số lớp:

$$\text{pos_weight}_c = \frac{\text{số mẫu âm}_c}{\text{số mẫu dương}_c}$$

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C w_c \cdot [y_{ic} \log(\sigma(x_{ic})) + (1 - y_{ic}) \log(1 - \sigma(x_{ic}))]$$

Clip trọng số: Giới hạn tối đa ở **50x** để tránh over-penalizing lớp đa số, cải thiện precision.

AdamW Optimizer

AdamW = Adam + **Weight Decay tách biệt** (decoupled).

- **Adam thường:** Weight decay ghép vào gradient $\rightarrow$ không hiệu quả với adaptive learning rate
- **AdamW:** Áp dụng weight decay **trực tiếp** lên tham số sau bước cập nhật

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \cdot \lambda \cdot \theta_t$$

- η : learning rate (2×10^{-3})
- λ : weight decay (10^{-4})
- $\hat{m}_t, \hat{v}_t$: bias-corrected first/second moment

Gradient Clipping

Vấn đề: Exploding gradient – gradient quá lớn làm cập nhật tham số mất ổn định.

Giải pháp: Cắt gradient nếu chuẩn vượt ngưỡng:

$$\text{Nếu } \|\mathbf{g}\|_2 > \text{max_norm} \Rightarrow \mathbf{g} \leftarrow \mathbf{g} \cdot \frac{\text{max_norm}}{\|\mathbf{g}\|_2}$$

- `max_norm = 1.0`
- Áp dụng trước `optimizer.step()`
- Đặc biệt quan trọng với RNN/LSTM (chuỗi dài)

Warmup Cosine LR Scheduler

Warmup phase: Tăng learning rate tuyến tính từ 0 $\rightarrow$ LR tối đa trong **10%** tổng số bước.

Cosine decay phase: Giảm learning rate theo hàm cosine:

$$\text{LR}(\text{step}) = \text{LR}_{\min} + (\text{LR}_{\max} - \text{LR}_{\min}) \cdot \frac{1}{2} \left(1 + \cos \left(\pi \cdot \frac{\text{step} - \text{warmup}}{\text{total} - \text{warmup}} \right) \right)$$

- **Warmup:** Tránh cập nhật quá mạnh ở đầu huấn luyện
- **Cosine decay:** Giảm dần mượt mà $\rightarrow$ hội tụ tốt hơn

Focal Loss

Vấn đề: Class-weighted BCE vẫn chưa tập trung đủ vào các mẫu khó phân loại (các lớp thiểu số).

Giải pháp: Focal Loss — giảm trọng số mẫu dễ, tập trung vào mẫu khó:

$$\text{FL}(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

- $\gamma = 2$: focusing parameter — càng cao, trọng số mẫu dễ càng giảm
- α_t : class weight — cân bằng giữa các lớp
- Đặc biệt hiệu quả trên các lớp thiểu số (threat, identity_hate)

Early Stopping

Giám sát: Validation ROC-AUC (macro) – metric chính của bài toán.

- **Patience:** 2 epochs không cải thiện → dừng
- **Mode:** Maximize (theo dõi ROC-AUC, không phải loss)
- **Checkpoint:** Lưu lại state của mô hình tốt nhất, restore khi dừng

Tại sao dùng AUC thay vì loss?

- Class-weighted loss bị ảnh hưởng bởi trọng số → không phản ánh đúng chất lượng
- ROC-AUC đo khả năng phân biệt thực sự, bất kể threshold

Optimal Threshold Search

Vấn đề: Threshold mặc định 0.5 không tối ưu cho dữ liệu mất cân bằng.

Giải pháp: Tìm threshold tối ưu **cho từng lớp** riêng biệt:

- Duyệt threshold từ 0.1 đến 0.9 (bước 0.01)
- Chọn threshold cho F1-score cao nhất trên validation set
- Mỗi lớp có threshold khác nhau phù hợp với phân phối riêng

Kết quả: Cải thiện đáng kể precision (giảm false positives) trong khi vẫn giữ recall cao.

Các cải tiến về code

- **Masked Pooling:** Bỏ qua padding tokens khi tính mean/max pooling — tránh làm loãng đặc trưng
- **AMP (Automatic Mixed Precision):** Giảm bộ nhớ GPU $\times 2$, tăng tốc $\sim 1.5\times$
- **GloVe 300D:** Embedding đã được huấn luyện trước thay vì khởi tạo ngẫu nhiên — hội tụ nhanh hơn
- **Pre-LN Transformer:** LayerNorm đặt trước attention/FFN thay vì sau — huấn luyện ổn định hơn, cho phép 4 tầng mã hóa
- **CLS token:** Thay thế mean pooling bằng [CLS] token — Transformer cải thiện đáng kể F1 Macro

- 1 Giới thiệu
- 2 Cơ sở lý thuyết
- 3 Phương pháp tối ưu
- 4 Kết quả và nhận xét

So sánh tổng quan các mô hình (full 159K mẫu)

Mô hình	Params	Best AUC	F1 Macro	F1 Micro
Attn BiLSTM	6.5M	0.9834	0.6311	0.7324
RCNN	6.6M	0.9839	0.5689	0.6655
Attn LSTM	4.9M	0.9812	0.5650	0.6820
BiLSTM	6.3M	0.9800	0.5520	0.6689
LSTM	4.8M	0.9786	0.5459	0.6658
GRU	4.6M	0.9805	0.5146	0.6198
Transformer	4.3M	0.9789	0.4625	0.5947

Best AUC: RCNN (0.9839) **Best F1 Macro:** Attention BiLSTM (0.6311)

Kết quả chi tiết – RCNN (Best AUC = 0.9839)

Nhãn	Precision	Recall	F1	AUC
toxic	0.62	0.89	0.73	0.9753
severe_toxic	0.25	0.96	0.40	0.9897
obscene	0.76	0.87	0.81	0.9892
threat	0.29	0.79	0.43	0.9878
insult	0.55	0.92	0.69	0.9841
identity_hate	0.20	0.85	0.32	0.9776
Macro Avg	0.44	0.88	0.56	0.9839

Với optimal thresholds: F1 Macro = 0.63, F1 Micro = 0.74

Nhận xét: Precision cải thiện mạnh so với lần chạy trước (10K mẫu) – dữ liệu lớn giúp mô hình học đặc trưng tốt hơn.

Kết quả chi tiết – Attention BiLSTM (Best F1 Macro = 0.63)

Nhãn	Precision	Recall	F1	AUC
toxic	0.67	0.87	0.76	0.9749
severe_toxic	0.29	0.91	0.44	0.9897
obscene	0.62	0.94	0.75	0.9901
threat	0.37	0.68	0.48	0.9816
insult	0.54	0.92	0.68	0.9845
identity_hate	0.32	0.73	0.45	0.9800
Macro Avg	0.47	0.84	0.59	0.9834

Với optimal thresholds: F1 Macro = 0.65, F1 Micro = 0.75

Nhận xét: F1 Macro cao nhất – attention giúp cân bằng precision/recall tốt hơn so với mean pooling.

Kết quả chi tiết – BiLSTM

Nhãn	Precision	Recall	F1	AUC
toxic	0.60	0.89	0.71	0.9736
severe_toxic	0.21	0.97	0.35	0.9880
obscene	0.58	0.94	0.72	0.9898
threat	0.13	0.73	0.22	0.9734
insult	0.48	0.93	0.63	0.9812
identity_hate	0.14	0.84	0.23	0.9741
Macro Avg	0.36	0.88	0.48	0.9800

Với optimal thresholds: F1 Macro = 0.58, F1 Micro = 0.71

Nhận xét: Recall rất cao (0.88) nhưng precision thấp hơn Attention BiLSTM → chứng tỏ attention thực sự giúp lọc nhiễu.

Kết quả tối ưu hóa trên Jigsaw (3 mô hình)

Mô hình	Phiên bản	AUC	F1 Macro	F1 Micro
RCNN	Gốc	0.9839	0.5689	0.6655
RCNN	Tối ưu	0.9846	0.6530	0.7703
AttnBiLSTM	Gốc	0.9834	0.6311	0.7324
AttnBiLSTM	Tối ưu	0.9833	0.6344	0.7608
Transformer	Gốc	0.9789	0.4625	0.5947
Transformer	Tối ưu	0.9797	0.6274	0.7497

Cải thiện nổi bật: Transformer F1 Macro +35.7% (0.4625 → 0.6274)

Kết quả trên HateXplain (3 mô hình)

Mô hình	AUC	F1 Macro	F1 Micro	Precision	Recall
RCNN	0.8156	0.6399	0.6571	0.6484	0.6414
Transformer	0.8087	0.6375	0.6402	0.6415	0.6356
AttnBiLSTM	0.8039	0.6169	0.6380	0.6239	0.6229

- AUC thấp hơn Jigsaw (0.80 vs 0.98) — dữ liệu khó hơn, số mẫu ít hơn (20K vs 160K)
- Transformer F1 Macro gần tương đương RCNN — khác biệt rõ so với kết quả trên Jigsaw

Thảo luận về hiệu suất Transformer

- **Trên Jigsaw (đa nhân):**
 - Gốc (Post-LN, mean pool, 2 tầng): F1 Macro = 0.4625 (thấp nhất trong 7 mô hình)
 - Tối ưu (Pre-LN, CLS token, 4 tầng): F1 Macro = 0.6274 (+35.7%)
 - Khoảng cách với RCNN thu hẹp từ 0.1064 $\rightarrow$ 0.0256
- **Trên HateXplain (đa lớp):**
 - Transformer: F1 Macro = 0.6375 $\approx$ RCNN (0.6399)
 - Vượt trội so với AttnBiLSTM (0.6169)

Kết luận: Transformer không yếu hơn RNN về bản chất — hiệu suất phụ thuộc vào cấu hình và đặc thù bài toán.

Nhận xét chung

- **Tối ưu hóa cải thiện đáng kể:** Transformer F1 Macro +35.7%, RCNN +14.8% trên Jigsaw
- **RCNN dẫn đầu trên cả hai tập:** AUC 0.9846 (Jigsaw), 0.8156 (HateXplain) — BiLSTM + max pooling rất hiệu quả
- **Transformer cạnh tranh sau tối ưu:** Pre-LN + CLS + 4 tầng thu hẹp khoảng cách với RCNN từ 0.1064 → 0.0256
- **Hiệu suất phụ thuộc bài toán:** Transformer kém trên Jigsaw (đa nhãn) nhưng gần tương đương RCNN trên HateXplain (đơn lớp)
- **GloVe + Masked Pooling cải thiện ổn định:** Tất cả mô hình đều hưởng lợi từ embedding có sẵn và pooling đúng
- **Thách thức còn lại:** Precision thấp ở lớp thiểu số (threat, identity_hate)

Hướng phát triển

- **Tinh chỉnh Focal Loss:** Tối ưu tham số γ và α cho từng mô hình
- **Ensemble:** Kết hợp predictions từ RCNN + Transformer
- **Data augmentation:** Back-translation, synonym replacement cho lớp thiểu số
- **Hyperparameter tuning:** Grid search / Bayesian optimization cho từng mô hình
- **HateXplain bản đầy đủ:** Sử dụng 5-fold cross-validation thay vì split đơn
- **Mở rộng mô hình:** Áp dụng cấu hình tối ưu cho toàn bộ 7 mô hình ban đầu

Kết luận

- Đã cài đặt từ gốc **3 kiến trúc** (RCNN, Attention BiLSTM, Transformer) + đánh giá chéo trên 2 tập dữ liệu
- **Tối ưu hóa:** Masked Pooling, GloVe, AMP, Pre-LN, Focal Loss — cải thiện F1 Macro lên đến +35.7%
- **Kết quả Jigsaw:** RCNN AUC 0.9846, F1 Macro 0.6530; Transformer F1 Macro 0.6274
- **Kết quả HateXplain:** RCNN AUC 0.8156, F1 Macro 0.6399; Transformer gần tương đương (0.6375)
- **Phát hiện chính:** Transformer không kém hơn RNN — hiệu suất phụ thuộc vào cấu hình và bài toán
- **Thách thức:** Mất cân bằng lớp — precision thấp ở các lớp thiểu số

Cảm ơn thầy đã lắng nghe!

Tài liệu tham khảo I

- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Cho, K., et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *EMNLP*.
- Lai, S., et al. (2015). Recurrent convolutional neural networks for text classification. *AAAI*.
- Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS*.
- Bahdanau, D., et al. (2014). Neural machine translation by jointly learning to align and translate. *ICLR*.
- Loshchilov, I., & Hutter, F. (2017). Decoupled weight decay regularization. *ICLR*.
- Pennington, J., et al. (2014). GloVe: Global vectors for word representation. *EMNLP*.

Tài liệu tham khảo II

Mathew, B., et al. (2021). HateXplain: A benchmark dataset for hate speech detection. *ICLR*.